

```

//+-----
//|                                     RectangleZoneAnalyzer_03.mq4
//|
//|  Блок 03: статистика касаний по возрасту и параметрам паттерна.
//|
//|  Вход : touches.csv
//|  Выход: age_statistics.csv
//|          pattern_statistics.csv
//|
//|  В расчёт входят только касания:
//|  TouchStatus=TOUCH и EntryDirection=EXPECTED_SIDE.
//+-----
#property copyright "Copyright 2026"
#property version   "1.00"
#property strict
#property script_show_inputs

//--- Файлы находятся в MQL4\Files
input string InputTouchesFileName           = "touches.csv";
input string OutputAgeStatisticsFileName    =
"age_statistics.csv";
input string OutputPatternStatisticsFileName =
"pattern_statistics.csv";

//--- Обычно touches.csv относится к одному графику.
//--- true: использовать только текущие Symbol() и Period().
input bool   FilterCurrentChartOnly         = true;

//--- Минимальная выборка для признака Reliable=YES
input int    MinReliableSamples             = 30;

//--- Границы торговых сессий по серверному времени терминала
//--- 00:00..AsiaEndHour           = ASIA
//--- AsiaEndHour..LondonEnd       = LONDON
//--- LondonEnd..NewYorkEnd        = NEW_YORK
//--- NewYorkEnd..24:00            = OTHER
input int    AsiaEndHour             = 7;
input int    LondonEndHour           = 13;
input int    NewYorkEndHour          = 21;

```

```
//--- Печать прогресса; 0 отключает сообщения
input int      ProgressEveryRows      = 5000;
```

```
#define TOUCHES_CSV_COLUMNS 137
#define AGE_BASE_COUNT      2
#define ZONE_GROUP_COUNT    3
#define AGE_BUCKET_COUNT    9
#define TOUCH_BUCKET_COUNT  5
#define DEPTH_BUCKET_COUNT  4
#define APPROACH_BUCKET_COUNT 4
#define HEIGHT_BUCKET_COUNT 5
#define SESSION_BUCKET_COUNT 4
```

```
//+-----
//| Накопленная статистика группы
```

```
//+-----
struct StatAccumulator
{
    int total;

    int localReversalFirst;
    int localBreakoutFirst;
    int localTimeout;
    int localOther;

    int primaryReversal;
    int localBreakout;

    int sequenceReversalOnly;
    int sequenceBreakoutOnly;
    int sequenceReversalThenBreakout;
    int sequenceBreakoutThenReversal;
    int sequenceNone;
    int sequenceOther;

    int reached05ATR;
    int reached10ATR;
    int reached15ATR;
    int reached20ATR;
    int reached1Height;
    int reached2Height;
    int reached3Height;

    int structuralOppositeFirst;
    int structuralSameTypeFirst;
    int structuralTimeout;
    int structuralTargetsBroken;
    int structuralEndOfHistory;
    int structuralOther;
```

```

double sumMFE_CloseATR;
int countMFE_CloseATR;
double sumMFE_CloseHeights;
int countMFE_CloseHeights;
double sumMAE_CloseATR;
int countMAE_CloseATR;
double sumMAE_CloseHeights;
int countMAE_CloseHeights;

double sumBarsPrimaryReversal;
int countBarsPrimaryReversal;
double sumBarsLocalBreakout;
int countBarsLocalBreakout;
double sumBarsStructuralReversal;
int countBarsStructuralReversal;
double sumBarsStructuralBreakout;
int countBarsStructuralBreakout;

double sumAgeMinutes;
int countAgeMinutes;
double sumAgeBars;
int countAgeBars;
double sumTouchNumber;
int countTouchNumber;
double sumDepthPercent;
int countDepthPercent;
double sumApproachNetMoveATR5;
int countApproachNetMoveATR5;
double sumApproachRangeATR5;
int countApproachRangeATR5;
double sumZoneHeightATR;
int countZoneHeightATR;
double sumTouchBodyATR;
int countTouchBodyATR;
double sumTouchRelativeVolume;
int countTouchRelativeVolume;
};

//--- AGE_ONLY: basis × zone × age
StatAccumulator g_ageStats[];

//--- AGE_TOUCH: basis × zone × age × touch
StatAccumulator g_ageTouchStats[];

//--- AGE_DEPTH: basis × zone × age × depth
StatAccumulator g_ageDepthStats[];

//--- AGE_TOUCH_DEPTH: basis × zone × age × touch × depth
StatAccumulator g_ageTouchDepthStats[];

//--- AGE_APPROACH5: basis × zone × age × approach
StatAccumulator g_ageApproachStats[];

```

```

//--- AGE_ZONE_HEIGHT: basis × zone × age × height
StatAccumulator g_ageHeightStats[];

//--- AGE_SESSION: basis × zone × age × session
StatAccumulator g_ageSessionStats[];

StatAccumulator g_globalStats;

//--- Индексы обязательных колонок touches.csv
int g_idxSymbol = -1;
int g_idxTimeframe = -1;
int g_idxZoneType = -1;
int g_idxTouchNumber = -1;
int g_idxTouchStatus = -1;
int g_idxEntryDirection = -1;
int g_idxTouchDecisionTime = -1;
int g_idxAgeCalendarMinutes = -1;
int g_idxAgeBars = -1;
int g_idxAgeTradingHours = -1;
int g_idxZoneHeightATR = -1;
int g_idxTouchDepthPercent = -1;
int g_idxTouchBodyATR = -1;
int g_idxTouchRelativeVolume = -1;
int g_idxApproachNetMoveATR5 = -1;
int g_idxApproachRangeATR5 = -1;
int g_idxReached05ATR = -1;
int g_idxReached10ATR = -1;
int g_idxReached15ATR = -1;
int g_idxReached20ATR = -1;
int g_idxReached1Height = -1;
int g_idxReached2Height = -1;
int g_idxReached3Height = -1;
int g_idxPrimaryReversal = -1;
int g_idxBarsPrimaryReversal = -1;
int g_idxLocalBreakout = -1;
int g_idxBarsLocalBreakout = -1;
int g_idxLocalFirstResult = -1;
int g_idxLocalSequence = -1;
int g_idxStructuralReversal = -1;
int g_idxBarsStructuralReversal = -1;
int g_idxStructuralBreakout = -1;
int g_idxBarsStructuralBreakout = -1;
int g_idxStructuralFirstResult = -1;
int g_idxMFE_CloseATR = -1;
int g_idxMFE_CloseHeights = -1;
int g_idxMAE_CloseATR = -1;
int g_idxMAE_CloseHeights = -1;

//--- Счётчики чтения
int g_rowsRead = 0;
int g_rowsIncluded = 0;
int g_skippedWrongSide = 0;

```

```
int g_skippedOtherChart    = 0;
int g_skippedInvalid       = 0;
```

```
//+-----
```

```
//| Строковое имя таймфрейма
```

```
//+-----
```

```
string TimeframeToString(int timeframe)
```

```
{
    switch(timeframe)
    {
        case PERIOD_M1:    return("M1");
        case PERIOD_M5:    return("M5");
        case PERIOD_M15:   return("M15");
        case PERIOD_M30:   return("M30");
        case PERIOD_H1:    return("H1");
        case PERIOD_H4:    return("H4");
        case PERIOD_D1:    return("D1");
        case PERIOD_W1:    return("W1");
        case PERIOD_MN1:   return("MN1");
    }

    return(IntegerToString(timeframe));
}
```

```
//+-----
```

```
//| Очистка накопителя
```

```
//+-----
```

```
void ResetAccumulator(StatAccumulator &stat)
```

```
{
    stat.total = 0;

    stat.localReversalFirst = 0;
    stat.localBreakoutFirst = 0;
    stat.localTimeout = 0;
    stat.localOther = 0;

    stat.primaryReversal = 0;
    stat.localBreakout = 0;

    stat.sequenceReversalOnly = 0;
    stat.sequenceBreakoutOnly = 0;
    stat.sequenceReversalThenBreakout = 0;
    stat.sequenceBreakoutThenReversal = 0;
    stat.sequenceNone = 0;
    stat.sequenceOther = 0;
}
```

```

stat.reached05ATR = 0;
stat.reached10ATR = 0;
stat.reached15ATR = 0;
stat.reached20ATR = 0;
stat.reached1Height = 0;
stat.reached2Height = 0;
stat.reached3Height = 0;

stat.structuralOppositeFirst = 0;
stat.structuralSameTypeFirst = 0;
stat.structuralTimeout = 0;
stat.structuralTargetsBroken = 0;
stat.structuralEndOfHistory = 0;
stat.structuralOther = 0;

stat.sumMFE_CloseATR = 0.0;
stat.countMFE_CloseATR = 0;
stat.sumMFE_CloseHeights = 0.0;
stat.countMFE_CloseHeights = 0;
stat.sumMAE_CloseATR = 0.0;
stat.countMAE_CloseATR = 0;
stat.sumMAE_CloseHeights = 0.0;
stat.countMAE_CloseHeights = 0;

stat.sumBarsPrimaryReversal = 0.0;
stat.countBarsPrimaryReversal = 0;
stat.sumBarsLocalBreakout = 0.0;
stat.countBarsLocalBreakout = 0;
stat.sumBarsStructuralReversal = 0.0;
stat.countBarsStructuralReversal = 0;
stat.sumBarsStructuralBreakout = 0.0;
stat.countBarsStructuralBreakout = 0;

stat.sumAgeMinutes = 0.0;
stat.countAgeMinutes = 0;
stat.sumAgeBars = 0.0;
stat.countAgeBars = 0;
stat.sumTouchNumber = 0.0;
stat.countTouchNumber = 0;
stat.sumDepthPercent = 0.0;
stat.countDepthPercent = 0;
stat.sumApproachNetMoveATR5 = 0.0;
stat.countApproachNetMoveATR5 = 0;
stat.sumApproachRangeATR5 = 0.0;
stat.countApproachRangeATR5 = 0;
stat.sumZoneHeightATR = 0.0;
stat.countZoneHeightATR = 0;
stat.sumTouchBodyATR = 0.0;
stat.countTouchBodyATR = 0;
stat.sumTouchRelativeVolume = 0.0;
stat.countTouchRelativeVolume = 0;
}

```

```

//+-----
//| Инициализация массивов статистики
//+-----
bool InitializeStatistics()
{
    int ageSize =
        AGE_BASE_COUNT * ZONE_GROUP_COUNT * AGE_BUCKET_COUNT;

    int ageTouchSize =
        ageSize * TOUCH_BUCKET_COUNT;

    int ageDepthSize =
        ageSize * DEPTH_BUCKET_COUNT;

    int ageTouchDepthSize =
        ageSize * TOUCH_BUCKET_COUNT * DEPTH_BUCKET_COUNT;

    int ageApproachSize =
        ageSize * APPROACH_BUCKET_COUNT;

    int ageHeightSize =
        ageSize * HEIGHT_BUCKET_COUNT;

    int ageSessionSize =
        ageSize * SESSION_BUCKET_COUNT;

    if(ArrayResize(g_ageStats, ageSize) != ageSize)
        return(false);

    if(ArrayResize(g_ageTouchStats, ageTouchSize) !=
ageTouchSize)
        return(false);

    if(ArrayResize(g_ageDepthStats, ageDepthSize) !=
ageDepthSize)
        return(false);

    if(ArrayResize(g_ageTouchDepthStats, ageTouchDepthSize) !=
ageTouchDepthSize)
        return(false);

    if(ArrayResize(g_ageApproachStats, ageApproachSize) !=
ageApproachSize)
        return(false);

    if(ArrayResize(g_ageHeightStats, ageHeightSize) !=
ageHeightSize)
        return(false);
}

```

```

    if(ArrayResize(g_ageSessionStats, ageSessionSize) !=
ageSessionSize)
        return(false);

    for(int i = 0; i < ArraySize(g_ageStats); i++)
        ResetAccumulator(g_ageStats[i]);

    for(int j = 0; j < ArraySize(g_ageTouchStats); j++)
        ResetAccumulator(g_ageTouchStats[j]);

    for(int k = 0; k < ArraySize(g_ageDepthStats); k++)
        ResetAccumulator(g_ageDepthStats[k]);

    for(int m = 0; m < ArraySize(g_ageTouchDepthStats); m++)
        ResetAccumulator(g_ageTouchDepthStats[m]);

    for(int n = 0; n < ArraySize(g_ageApproachStats); n++)
        ResetAccumulator(g_ageApproachStats[n]);

    for(int p = 0; p < ArraySize(g_ageHeightStats); p++)
        ResetAccumulator(g_ageHeightStats[p]);

    for(int q = 0; q < ArraySize(g_ageSessionStats); q++)
        ResetAccumulator(g_ageSessionStats[q]);

    ResetAccumulator(g_globalStats);
    return(true);
}

```

//+-----

//| ИНДЕКСЫ ПЛОСКИХ МАССИВОВ

//+-----

```

int BaseAgeIndex(int ageBasis, int zoneGroup, int ageBucket)
{
    return(
        (ageBasis * ZONE_GROUP_COUNT + zoneGroup) *
        AGE_BUCKET_COUNT + ageBucket
    );
}

```

```

int AgeTouchIndex(
    int ageBasis,
    int zoneGroup,
    int ageBucket,
    int touchBucket
)
{
    return(
        BaseAgeIndex(ageBasis, zoneGroup, ageBucket) *

```

```

        TOUCH_BUCKET_COUNT + touchBucket
    );
}

int AgeDepthIndex(
    int ageBasis,
    int zoneGroup,
    int ageBucket,
    int depthBucket
)
{
    return(
        BaseAgeIndex(ageBasis, zoneGroup, ageBucket) *
        DEPTH_BUCKET_COUNT + depthBucket
    );
}

int AgeTouchDepthIndex(
    int ageBasis,
    int zoneGroup,
    int ageBucket,
    int touchBucket,
    int depthBucket
)
{
    return(
        (
            BaseAgeIndex(ageBasis, zoneGroup, ageBucket) *
            TOUCH_BUCKET_COUNT + touchBucket
        ) * DEPTH_BUCKET_COUNT + depthBucket
    );
}

int AgeApproachIndex(
    int ageBasis,
    int zoneGroup,
    int ageBucket,
    int approachBucket
)
{
    return(
        BaseAgeIndex(ageBasis, zoneGroup, ageBucket) *
        APPROACH_BUCKET_COUNT + approachBucket
    );
}

int AgeHeightIndex(
    int ageBasis,
    int zoneGroup,
    int ageBucket,
    int heightBucket
)

```

```

{
    return(
        BaseAgeIndex(ageBasis, zoneGroup, ageBucket) *
        HEIGHT_BUCKET_COUNT + heightBucket
    );
}

```

```

int AgeSessionIndex(
    int ageBasis,
    int zoneGroup,
    int ageBucket,
    int sessionBucket
)
{
    return(
        BaseAgeIndex(ageBasis, zoneGroup, ageBucket) *
        SESSION_BUCKET_COUNT + sessionBucket
    );
}

```

//+-----

//| Поиск колонки по имени

//+-----

```

int FindColumn(string &headers[], string columnName)
{
    int count = ArraySize(headers);

    for(int i = 0; i < count; i++)
    {
        if(headers[i] == columnName)
            return(i);
    }

    return(-1);
}

```

//+-----

//| Назначение индексов колонок

//+-----

```

bool MapRequiredColumns(string &headers[])
{
    g_idxSymbol = FindColumn(headers, "Symbol"
);
    g_idxTimeframe = FindColumn(headers,
    "Timeframe");
}

```

```

    g_idxZoneType                = FindColumn(headers, "ZoneType"
);
    g_idxTouchNumber              = FindColumn(headers,
"TouchNumber");
    g_idxTouchStatus              = FindColumn(headers,
"TouchStatus");
    g_idxEntryDirection           = FindColumn(headers,
"EntryDirection");
    g_idxTouchDecisionTime        = FindColumn(headers,
"TouchDecisionTime");
    g_idxAgeCalendarMinutes       = FindColumn(headers,
"ZoneAgeCalendarMinutes");
    g_idxAgeBars                  = FindColumn(headers,
"ZoneAgeBars");
    g_idxAgeTradingHours          = FindColumn(headers,
"ZoneAgeTradingHours");
    g_idxZoneHeightATR            = FindColumn(headers,
"ZoneHeightATR");
    g_idxTouchDepthPercent        = FindColumn(headers,
"TouchDepthPercent");
    g_idxTouchBodyATR             = FindColumn(headers,
"TouchBodyATR");
    g_idxTouchRelativeVolume      = FindColumn(headers,
"TouchRelativeVolume");
    g_idxApproachNetMoveATR5       = FindColumn(headers,
"ApproachNetMoveATR_5");
    g_idxApproachRangeATR5        = FindColumn(headers,
"ApproachRangeATR_5");
    g_idxReached05ATR             = FindColumn(headers,
"Reached_0_5_ATR");
    g_idxReached10ATR             = FindColumn(headers,
"Reached_1_0_ATR");
    g_idxReached15ATR             = FindColumn(headers,
"Reached_1_5_ATR");
    g_idxReached20ATR             = FindColumn(headers,
"Reached_2_0_ATR");
    g_idxReached1Height           = FindColumn(headers,
"Reached_1_ZoneHeight");
    g_idxReached2Height           = FindColumn(headers,
"Reached_2_ZoneHeight");
    g_idxReached3Height           = FindColumn(headers,
"Reached_3_ZoneHeight");
    g_idxPrimaryReversal          = FindColumn(headers,
"PrimaryReversal");
    g_idxBarsPrimaryReversal      = FindColumn(headers,
"BarsToPrimaryReversal");
    g_idxLocalBreakout            = FindColumn(headers,
"LocalBreakout");
    g_idxBarsLocalBreakout        = FindColumn(headers,
"BarsToLocalBreakout");
    g_idxLocalFirstResult         = FindColumn(headers,
"LocalFirstResult");

```

```

    g_idxLocalSequence           = FindColumn(headers,
"LocalSequence");
    g_idxStructuralReversal      = FindColumn(headers,
"StructuralReversal");
    g_idxBarsStructuralReversal = FindColumn(headers,
"BarsToStructuralReversal");
    g_idxStructuralBreakout      = FindColumn(headers,
"StructuralBreakout");
    g_idxBarsStructuralBreakout = FindColumn(headers,
"BarsToStructuralBreakout");
    g_idxStructuralFirstResult   = FindColumn(headers,
"StructuralFirstResult");
    g_idxMFE_CloseATR            = FindColumn(headers,
"MFE_CloseATR");
    g_idxMFE_CloseHeights        = FindColumn(headers,
"MFE_CloseZoneHeights");
    g_idxMAE_CloseATR            = FindColumn(headers,
"MAE_CloseATR");
    g_idxMAE_CloseHeights        = FindColumn(headers,
"MAE_CloseZoneHeights");

    if(g_idxSymbol < 0 ||
        g_idxTimeframe < 0 ||
        g_idxZoneType < 0 ||
        g_idxTouchNumber < 0 ||
        g_idxTouchStatus < 0 ||
        g_idxEntryDirection < 0 ||
        g_idxTouchDecisionTime < 0 ||
        g_idxAgeCalendarMinutes < 0 ||
        g_idxAgeBars < 0 ||
        g_idxAgeTradingHours < 0 ||
        g_idxZoneHeightATR < 0 ||
        g_idxTouchDepthPercent < 0 ||
        g_idxTouchBodyATR < 0 ||
        g_idxTouchRelativeVolume < 0 ||
        g_idxApproachNetMoveATR5 < 0 ||
        g_idxApproachRangeATR5 < 0 ||
        g_idxReached05ATR < 0 ||
        g_idxReached10ATR < 0 ||
        g_idxReached15ATR < 0 ||
        g_idxReached20ATR < 0 ||
        g_idxReached1Height < 0 ||
        g_idxReached2Height < 0 ||
        g_idxReached3Height < 0 ||
        g_idxPrimaryReversal < 0 ||
        g_idxBarsPrimaryReversal < 0 ||
        g_idxLocalBreakout < 0 ||
        g_idxBarsLocalBreakout < 0 ||
        g_idxLocalFirstResult < 0 ||
        g_idxLocalSequence < 0 ||
        g_idxStructuralReversal < 0 ||
        g_idxBarsStructuralReversal < 0 ||

```

```

        g_idxStructuralBreakout < 0 ||
        g_idxBarsStructuralBreakout < 0 ||
        g_idxStructuralFirstResult < 0 ||
        g_idxMFE_CloseATR < 0 ||
        g_idxMFE_CloseHeights < 0 ||
        g_idxMAE_CloseATR < 0 ||
        g_idxMAE_CloseHeights < 0)
    {
        Print(
"Ошибка: в touches.csv отсутствуют обязательные колонки блока 03."
);
        return(false);
    }

    return(true);
}

```

//+-----

//| Проверка значения 0/1

//+-----

```

bool CsvBool(string value)
{
    return(value == "1" || value == "true" || value == "TRUE");
}

```

//+-----

//| Добавление необязательного числа

//+-----

```

void AddOptionalDouble(
    string value,
    double &sum,
    int &count
)
{
    if(StringLen(value) == 0)
        return;

    sum += StringToDouble(value);
    count++;
}

```

//+-----

//| Обновление одной статистической группы

//+-----

```

void UpdateAccumulator(
    StatAccumulator &stat,
    string &fields[],
    double ageMinutes,
    int touchNumber,
    double depthPercent,
    double approachNetMoveATR5,
    double approachRangeATR5,
    double zoneHeightATR
)
{
    stat.total++;

    string localFirst = fields[g_idxLocalFirstResult];

    if(localFirst == "REVERSAL_FIRST")
        stat.localReversalFirst++;
    else if(localFirst == "BREAKOUT_FIRST")
        stat.localBreakoutFirst++;
    else if(localFirst == "TIMEOUT")
        stat.localTimeout++;
    else
        stat.localOther++;

    if(CsvBool(fields[g_idxPrimaryReversal]))
        stat.primaryReversal++;

    if(CsvBool(fields[g_idxLocalBreakout]))
        stat.localBreakout++;

    string sequence = fields[g_idxLocalSequence];

    if(sequence == "REVERSAL_ONLY")
        stat.sequenceReversalOnly++;
    else if(sequence == "BREAKOUT_ONLY")
        stat.sequenceBreakoutOnly++;
    else if(sequence == "REVERSAL_THEN_BREAKOUT")
        stat.sequenceReversalThenBreakout++;
    else if(sequence == "BREAKOUT_THEN_REVERSAL")
        stat.sequenceBreakoutThenReversal++;
    else if(sequence == "NONE")
        stat.sequenceNone++;
    else
        stat.sequenceOther++;

    if(CsvBool(fields[g_idxReached05ATR]))
        stat.reached05ATR++;

    if(CsvBool(fields[g_idxReached10ATR]))
        stat.reached10ATR++;

    if(CsvBool(fields[g_idxReached15ATR]))

```

```

    stat.reached15ATR++;

    if(CsvBool(fields[g_idxReached20ATR]))
        stat.reached20ATR++;

    if(CsvBool(fields[g_idxReached1Height]))
        stat.reached1Height++;

    if(CsvBool(fields[g_idxReached2Height]))
        stat.reached2Height++;

    if(CsvBool(fields[g_idxReached3Height]))
        stat.reached3Height++;

    string structuralFirst = fields[g_idxStructuralFirstResult];

    if(structuralFirst == "OPPOSITE_ZONE_FIRST")
        stat.structuralOppositeFirst++;
    else if(structuralFirst == "SAME_TYPE_ZONE_FIRST")
        stat.structuralSameTypeFirst++;
    else if(structuralFirst == "TIMEOUT")
        stat.structuralTimeout++;
    else if(structuralFirst == "TARGETS_BROKEN")
        stat.structuralTargetsBroken++;
    else if(structuralFirst == "END_OF_HISTORY")
        stat.structuralEndOfHistory++;
    else
        stat.structuralOther++;

    AddOptionalDouble(
        fields[g_idxMFE_CloseATR],
        stat.sumMFE_CloseATR,
        stat.countMFE_CloseATR
    );

    AddOptionalDouble(
        fields[g_idxMFE_CloseHeights],
        stat.sumMFE_CloseHeights,
        stat.countMFE_CloseHeights
    );

    AddOptionalDouble(
        fields[g_idxMAE_CloseATR],
        stat.sumMAE_CloseATR,
        stat.countMAE_CloseATR
    );

    AddOptionalDouble(
        fields[g_idxMAE_CloseHeights],
        stat.sumMAE_CloseHeights,
        stat.countMAE_CloseHeights
    );

```

```

AddOptionalDouble (
    fields[g_idxBarsPrimaryReversal],
    stat.sumBarsPrimaryReversal,
    stat.countBarsPrimaryReversal
);

AddOptionalDouble (
    fields[g_idxBarsLocalBreakout],
    stat.sumBarsLocalBreakout,
    stat.countBarsLocalBreakout
);

AddOptionalDouble (
    fields[g_idxBarsStructuralReversal],
    stat.sumBarsStructuralReversal,
    stat.countBarsStructuralReversal
);

AddOptionalDouble (
    fields[g_idxBarsStructuralBreakout],
    stat.sumBarsStructuralBreakout,
    stat.countBarsStructuralBreakout
);

stat.sumAgeMinutes += ageMinutes;
stat.countAgeMinutes++;

AddOptionalDouble (
    fields[g_idxAgeBars],
    stat.sumAgeBars,
    stat.countAgeBars
);

stat.sumTouchNumber += touchNumber;
stat.countTouchNumber++;

stat.sumDepthPercent += depthPercent;
stat.countDepthPercent++;

stat.sumApproachNetMoveATR5 += approachNetMoveATR5;
stat.countApproachNetMoveATR5++;

stat.sumApproachRangeATR5 += approachRangeATR5;
stat.countApproachRangeATR5++;

stat.sumZoneHeightATR += zoneHeightATR;
stat.countZoneHeightATR++;

AddOptionalDouble (
    fields[g_idxTouchBodyATR],
    stat.sumTouchBodyATR,
    stat.countTouchBodyATR
);

```

```

);

AddOptionalDouble(
    fields[g_idxTouchRelativeVolume],
    stat.sumTouchRelativeVolume,
    stat.countTouchRelativeVolume
);
}

```

```

//+-----
//| Возрастная группа

```

```

//+-----
int GetAgeBucket(double ageMinutes)
{
    if(ageMinutes < 15.0)    return(0);
    if(ageMinutes < 30.0)    return(1);
    if(ageMinutes < 60.0)    return(2);
    if(ageMinutes < 120.0)   return(3);
    if(ageMinutes < 240.0)   return(4);
    if(ageMinutes < 480.0)   return(5);
    if(ageMinutes < 1440.0)  return(6);
    if(ageMinutes < 2880.0)  return(7);
    return(8);
}

```

```

string AgeBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("00_0_15_MIN");
        case 1: return("01_15_30_MIN");
        case 2: return("02_30_60_MIN");
        case 3: return("03_1_2_HOURS");
        case 4: return("04_2_4_HOURS");
        case 5: return("05_4_8_HOURS");
        case 6: return("06_8_24_HOURS");
        case 7: return("07_24_48_HOURS");
        case 8: return("08_48_HOURS_PLUS");
    }

    return("UNKNOWN");
}

```

```

//+-----
//| Группа номера касания

```

```

//+-----

```

```
int GetTouchBucket(int touchNumber)
{
    if(touchNumber <= 1) return(0);
    if(touchNumber == 2) return(1);
    if(touchNumber == 3) return(2);
    if(touchNumber == 4) return(3);
    return(4);
}
```

```
string TouchBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("TOUCH_1");
        case 1: return("TOUCH_2");
        case 2: return("TOUCH_3");
        case 3: return("TOUCH_4");
        case 4: return("TOUCH_5_PLUS");
    }

    return("ALL");
}
```

//+-----

//| Группа глубины закрытия

//+-----

```
int GetDepthBucket(double depthPercent)
{
    if(depthPercent < 25.0) return(0);
    if(depthPercent < 50.0) return(1);
    if(depthPercent < 75.0) return(2);
    return(3);
}
```

```
string DepthBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("DEPTH_0_25_PCT");
        case 1: return("DEPTH_25_50_PCT");
        case 2: return("DEPTH_50_75_PCT");
        case 3: return("DEPTH_75_100_PCT");
    }

    return("ALL");
}
```

//+-----

```
//| Скорость подхода за 5 баров
```

```
//+-----  
int GetApproachBucket(double approachNetMoveATR5)  
{  
    if(approachNetMoveATR5 <= 0.0) return(0);  
    if(approachNetMoveATR5 < 0.5) return(1);  
    if(approachNetMoveATR5 < 1.0) return(2);  
    return(3);  
}
```

```
string ApproachBucketName(int bucket)  
{  
    switch(bucket)  
    {  
        case 0: return("APPROACH_FLAT_OR_AWAY");  
        case 1: return("APPROACH_SLOW_LT_0_5_ATR");  
        case 2: return("APPROACH_MEDIUM_0_5_1_ATR");  
        case 3: return("APPROACH_FAST_GE_1_ATR");  
    }  
  
    return("ALL");  
}
```

```
//+-----  
//| Размер зоны относительно ATR
```

```
//+-----  
int GetHeightBucket(double zoneHeightATR)  
{  
    if(zoneHeightATR < 0.25) return(0);  
    if(zoneHeightATR < 0.50) return(1);  
    if(zoneHeightATR < 0.75) return(2);  
    if(zoneHeightATR < 1.00) return(3);  
    return(4);  
}
```

```
string HeightBucketName(int bucket)  
{  
    switch(bucket)  
    {  
        case 0: return("HEIGHT_LT_0_25_ATR");  
        case 1: return("HEIGHT_0_25_0_50_ATR");  
        case 2: return("HEIGHT_0_50_0_75_ATR");  
        case 3: return("HEIGHT_0_75_1_00_ATR");  
        case 4: return("HEIGHT_GE_1_00_ATR");  
    }  
  
    return("ALL");  
}
```

```

}

//+-----

//| Торговая сессия по серверному времени

//+-----
int GetSessionBucket(datetime decisionTime)
{
    int hour = TimeHour(decisionTime);

    if(hour < AsiaEndHour)
        return(0);

    if(hour < LondonEndHour)
        return(1);

    if(hour < NewYorkEndHour)
        return(2);

    return(3);
}

string SessionBucketName(int bucket)
{
    switch(bucket)
    {
        case 0: return("ASIA_SERVER_TIME");
        case 1: return("LONDON_SERVER_TIME");
        case 2: return("NEW_YORK_SERVER_TIME");
        case 3: return("OTHER_SERVER_TIME");
    }

    return("ALL");
}

string AgeBasisName(int basis)
{
    if(basis == 0)
        return("CALENDAR_TIME");

    return("TRADING_TIME");
}

string ZoneGroupName(int zoneGroup)
{
    if(zoneGroup == 1)
        return("BULL");

    if(zoneGroup == 2)
        return("BEAR");
}

```

```

    return("ALL");
}

int ZoneTypeToGroup(string zoneType)
{
    if(zoneType == "BULL")
        return(1);

    if(zoneType == "BEAR")
        return(2);

    return(-1);
}

//+-----

//| Обработка одной строки touches.csv

//+-----
bool ProcessTouchRow(string &fields[])
{
    if(fields[g_idxTouchStatus] != "TOUCH" ||
        fields[g_idxEntryDirection] != "EXPECTED_SIDE")
    {
        g_skippedWrongSide++;
        return(true);
    }

    if(FilterCurrentChartOnly)
    {
        if(fields[g_idxSymbol] != Symbol() ||
            fields[g_idxTimeframe] != TimeframeToString(Period()))
        {
            g_skippedOtherChart++;
            return(true);
        }
    }

    int zoneGroup = ZoneTypeToGroup(fields[g_idxZoneType]);

    if(zoneGroup < 0 ||
        StringLen(fields[g_idxTouchNumber]) == 0 ||
        StringLen(fields[g_idxTouchDecisionTime]) == 0 ||
        StringLen(fields[g_idxAgeCalendarMinutes]) == 0 ||
        StringLen(fields[g_idxAgeTradingHours]) == 0 ||
        StringLen(fields[g_idxTouchDepthPercent]) == 0 ||
        StringLen(fields[g_idxZoneHeightATR]) == 0 ||
        StringLen(fields[g_idxApproachNetMoveATR5]) == 0 ||
        StringLen(fields[g_idxApproachRangeATR5]) == 0)
    {
        g_skippedInvalid++;
    }
}

```

```

        return(true);
    }

    int touchNumber = (int)StringToInteger(fields[
g_idxTouchNumber]);
    datetime decisionTime = StringToTime(fields[
g_idxTouchDecisionTime]);

    double calendarAgeMinutes =
        StringToDouble(fields[g_idxAgeCalendarMinutes]);

    double tradingAgeMinutes =
        StringToDouble(fields[g_idxAgeTradingHours]) * 60.0;

    double depthPercent =
        StringToDouble(fields[g_idxTouchDepthPercent]);

    double approachNetMoveATR5 =
        StringToDouble(fields[g_idxApproachNetMoveATR5]);

    double approachRangeATR5 =
        StringToDouble(fields[g_idxApproachRangeATR5]);

    double zoneHeightATR =
        StringToDouble(fields[g_idxZoneHeightATR]);

    if(touchNumber < 1 ||
        decisionTime <= 0 ||
        calendarAgeMinutes < 0.0 ||
        tradingAgeMinutes < 0.0 ||
        depthPercent < 0.0 ||
        depthPercent > 100.0001 ||
        zoneHeightATR < 0.0)
    {
        g_skippedInvalid++;
        return(true);
    }

    int touchBucket = GetTouchBucket(touchNumber);
    int depthBucket = GetDepthBucket(depthPercent);
    int approachBucket = GetApproachBucket(approachNetMoveATR5);
    int heightBucket = GetHeightBucket(zoneHeightATR);
    int sessionBucket = GetSessionBucket(decisionTime);

    double ages[AGE_BASE_COUNT];
    ages[0] = calendarAgeMinutes;
    ages[1] = tradingAgeMinutes;

    //--- Общая статистика обновляется один раз на касание.
    UpdateAccumulator(
        g_globalStats,
        fields,

```

```

    calendarAgeMinutes,
    touchNumber,
    depthPercent,
    approachNetMoveATR5,
    approachRangeATR5,
    zoneHeightATR
);

//--- Для каждого типа возраста создаются строки ALL и BULL/BEAR.
for(int basis = 0; basis < AGE_BASE_COUNT; basis++)
{
    int ageBucket = GetAgeBucket(ages[basis]);

    for(int variant = 0; variant < 2; variant++)
    {
        int currentZoneGroup =
            variant == 0 ? 0 : zoneGroup;

        UpdateAccumulator(
            g_ageStats[
                BaseAgeIndex(
                    basis,
                    currentZoneGroup,
                    ageBucket
                )
            ],
            fields,
            ages[basis],
            touchNumber,
            depthPercent,
            approachNetMoveATR5,
            approachRangeATR5,
            zoneHeightATR
        );

        UpdateAccumulator(
            g_ageTouchStats[
                AgeTouchIndex(
                    basis,
                    currentZoneGroup,
                    ageBucket,
                    touchBucket
                )
            ],
            fields,
            ages[basis],
            touchNumber,
            depthPercent,
            approachNetMoveATR5,
            approachRangeATR5,
            zoneHeightATR

```

```
);

UpdateAccumulator(
    g_ageDepthStats[
        AgeDepthIndex(
            basis,
            currentZoneGroup,
            ageBucket,
            depthBucket
        )
    ],
    fields,
    ages[basis],
    touchNumber,
    depthPercent,
    approachNetMoveATR5,
    approachRangeATR5,
    zoneHeightATR
);
```

```
UpdateAccumulator(
    g_ageTouchDepthStats[
        AgeTouchDepthIndex(
            basis,
            currentZoneGroup,
            ageBucket,
            touchBucket,
            depthBucket
        )
    ],
    fields,
    ages[basis],
    touchNumber,
    depthPercent,
    approachNetMoveATR5,
    approachRangeATR5,
    zoneHeightATR
);
```

```
UpdateAccumulator(
    g_ageApproachStats[
        AgeApproachIndex(
            basis,
            currentZoneGroup,
            ageBucket,
            approachBucket
        )
    ],
    fields,
    ages[basis],
    touchNumber,
    depthPercent,
```

```

        approachNetMoveATR5,
        approachRangeATR5,
        zoneHeightATR
    );

    UpdateAccumulator(
        g_ageHeightStats[
            AgeHeightIndex(
                basis,
                currentZoneGroup,
                ageBucket,
                heightBucket
            )
        ],
        fields,
        ages[basis],
        touchNumber,
        depthPercent,
        approachNetMoveATR5,
        approachRangeATR5,
        zoneHeightATR
    );

    UpdateAccumulator(
        g_ageSessionStats[
            AgeSessionIndex(
                basis,
                currentZoneGroup,
                ageBucket,
                sessionBucket
            )
        ],
        fields,
        ages[basis],
        touchNumber,
        depthPercent,
        approachNetMoveATR5,
        approachRangeATR5,
        zoneHeightATR
    );
}

}

g_rowsIncluded++;
return(true);
}

```

//+-----

//| Чтение touches.csv

```

//+-----
bool LoadAndAggregateTouches()
{
    ResetLastError();

    int handle = FileOpen(
        InputTouchesFileName,
        FILE_READ | FILE_CSV | FILE_ANSI,
        ';'
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка открытия ",
            InputTouchesFileName,
            ". Код ошибки: ",
            GetLastError()
        );

        ResetLastError();
        return(false);
    }

    string headers[];
    ArrayResize(headers, TOUCHES_CSV_COLUMNS);

    for(int h = 0; h < TOUCHES_CSV_COLUMNS; h++)
    {
        if(FileIsEnding(handle))
        {
            Print(
                "Ошибка: touches.csv не содержит полного заголовка.");
            FileClose(handle);
            return(false);
        }

        headers[h] = FileReadString(handle);
    }

    if(!MapRequiredColumns(headers))
    {
        FileClose(handle);
        return(false);
    }

    string fields[];
    ArrayResize(fields, TOUCHES_CSV_COLUMNS);

    while(!FileIsEnding(handle))
    {

```

```

for(int c = 0; c < TOUCHES_CSV_COLUMNS; c++)
{
    if(FileIsEnding(handle) && c == 0)
    {
        fields[c] = "";
        break;
    }

    fields[c] = FileReadString(handle);
}

if(StringLen(fields[0]) == 0)
    break;

g_rowsRead++;
ProcessTouchRow(fields);

if(ProgressEveryRows > 0 &&
    g_rowsRead % ProgressEveryRows == 0)
{
    Print(
        "Блок 03: прочитано строк ",
        g_rowsRead,
        ", включено касаний ",
        g_rowsIncluded
    );
}
}

FileClose(handle);
return(true);
}

```

//+-----

//| Добавление поля в CSV-строку

//+-----

```

void AppendField(string &line, string value)
{
    if(StringLen(line) > 0)
        line += ";";

    line += value;
}

```

```

string DoubleOrBlank(double value, int digits)
{
    if(value == EMPTY_VALUE)
        return("");
}

```

```

    return(DoubleToString(value, digits));
}

string AverageOrBlank(double sum, int count, int digits)
{
    if(count <= 0)
        return("");

    return(DoubleToString(sum / count, digits));
}

string PercentOrBlank(int numerator, int denominator)
{
    if(denominator <= 0)
        return("");

    return(
        DoubleToString(
            100.0 * numerator / denominator,
            4
        )
    );
}

```

//+-----

//| Нижняя граница интервала Уилсона, 95%

//+-----

```

string WilsonLower95OrBlank(int success, int total)
{
    if(total <= 0)
        return("");

    double z = 1.95996398454005;
    double n = total;
    double p = success / n;
    double z2 = z * z;

    double center =
        p + z2 / (2.0 * n);

    double margin =
        z * MathSqrt(
            p * (1.0 - p) / n +
            z2 / (4.0 * n * n)
        );

    double lower =
        (center - margin) /
        (1.0 + z2 / n);
}

```

```

    if(lower < 0.0)
        lower = 0.0;

    return(DoubleToString(lower * 100.0, 4));
}

```

```

//+-----

```

```

//| Общий заголовок статистических файлов

```

```

//+-----

```

```

void WriteStatisticsHeader(int handle)
{

```

```

    string line = "";

```

```

    AppendField(line, "PatternType");
    AppendField(line, "AgeBasis");
    AppendField(line, "ScopeSymbol");
    AppendField(line, "ScopeTimeframe");
    AppendField(line, "ZoneType");
    AppendField(line, "AgeBucket");
    AppendField(line, "TouchBucket");
    AppendField(line, "DepthBucket");
    AppendField(line, "ApproachBucket");
    AppendField(line, "ZoneHeightBucket");
    AppendField(line, "SessionBucket");

```

```

    AppendField(line, "Samples");
    AppendField(line, "Reliable");
    AppendField(line, "MinReliableSamples");

```

```

    AppendField(line, "LocalReversalFirst");
    AppendField(line, "LocalBreakoutFirst");
    AppendField(line, "LocalTimeout");
    AppendField(line, "LocalOther");
    AppendField(line, "LocalDecisiveSamples");
    AppendField(line, "LocalReversalPctAll");
    AppendField(line, "LocalBreakoutPctAll");
    AppendField(line, "LocalTimeoutPctAll");
    AppendField(line, "LocalReversalPctDecisive");
    AppendField(line, "LocalReversalWilsonLower95");

```

```

    AppendField(line, "PrimaryReversal");
    AppendField(line, "PrimaryReversalPct");
    AppendField(line, "LocalBreakout");
    AppendField(line, "LocalBreakoutPct");

```

```

    AppendField(line, "SequenceReversalOnly");
    AppendField(line, "SequenceBreakoutOnly");
    AppendField(line, "SequenceReversalThenBreakout");
    AppendField(line, "SequenceBreakoutThenReversal");

```

```

AppendField(line, "SequenceNone");
AppendField(line, "SequenceOther");

AppendField(line, "Reached_0_5_ATR");
AppendField(line, "Reached_0_5_ATR_Pct");
AppendField(line, "Reached_1_0_ATR");
AppendField(line, "Reached_1_0_ATR_Pct");
AppendField(line, "Reached_1_5_ATR");
AppendField(line, "Reached_1_5_ATR_Pct");
AppendField(line, "Reached_2_0_ATR");
AppendField(line, "Reached_2_0_ATR_Pct");
AppendField(line, "Reached_1_ZoneHeight");
AppendField(line, "Reached_1_ZoneHeight_Pct");
AppendField(line, "Reached_2_ZoneHeight");
AppendField(line, "Reached_2_ZoneHeight_Pct");
AppendField(line, "Reached_3_ZoneHeight");
AppendField(line, "Reached_3_ZoneHeight_Pct");

AppendField(line, "StructuralOppositeFirst");
AppendField(line, "StructuralSameTypeFirst");
AppendField(line, "StructuralTimeout");
AppendField(line, "StructuralTargetsBroken");
AppendField(line, "StructuralEndOfHistory");
AppendField(line, "StructuralOther");
AppendField(line, "StructuralDecisiveSamples");
AppendField(line, "StructuralOppositePctAll");
AppendField(line, "StructuralSameTypePctAll");
AppendField(line, "StructuralOppositePctDecisive");
AppendField(line, "StructuralOppositeWilsonLower95");

AppendField(line, "AvgMFE_CloseATR");
AppendField(line, "AvgMAE_CloseATR");
AppendField(line, "AvgMFE_MAE_Ratio");
AppendField(line, "AvgMFE_CloseZoneHeights");
AppendField(line, "AvgMAE_CloseZoneHeights");

AppendField(line, "AvgBarsToPrimaryReversal");
AppendField(line, "AvgBarsToLocalBreakout");
AppendField(line, "AvgBarsToStructuralReversal");
AppendField(line, "AvgBarsToStructuralBreakout");

AppendField(line, "AvgAgeMinutes");
AppendField(line, "AvgAgeBars");
AppendField(line, "AvgTouchNumber");
AppendField(line, "AvgDepthPercent");
AppendField(line, "AvgApproachNetMoveATR_5");
AppendField(line, "AvgApproachRangeATR_5");
AppendField(line, "AvgZoneHeightATR");
AppendField(line, "AvgTouchBodyATR");
AppendField(line, "AvgTouchRelativeVolume");

FileWriteString(handle, line + "\r\n");

```

```
}
```

```
//+-----
```

```
//| Запись одной статистической строки
```

```
//+-----
```

```
void WriteStatisticsRow(  
    int handle,  
    string patternType,  
    int ageBasis,  
    int zoneGroup,  
    int ageBucket,  
    string touchBucket,  
    string depthBucket,  
    string approachBucket,  
    string heightBucket,  
    string sessionBucket,  
    StatAccumulator &stat  
)  
{  
    if(stat.total <= 0)  
        return;  
  
    string scopeSymbol =  
        FilterCurrentChartOnly ? Symbol() : "ALL";  
  
    string scopeTimeframe =  
        FilterCurrentChartOnly  
        ? TimeframeToString(Period())  
        : "ALL";  
  
    int localDecisive =  
        stat.localReversalFirst +  
        stat.localBreakoutFirst;  
  
    int structuralDecisive =  
        stat.structuralOppositeFirst +  
        stat.structuralSameTypeFirst;  
  
    double avgMFE = EMPTY_VALUE;  
    double avgMAE = EMPTY_VALUE;  
    double mfeMaeRatio = EMPTY_VALUE;  
  
    if(stat.countMFE_CloseATR > 0)  
        avgMFE =  
            stat.sumMFE_CloseATR /  
            stat.countMFE_CloseATR;  
  
    if(stat.countMAE_CloseATR > 0)  
        avgMAE =
```

```

        stat.sumMAE_CloseATR /
        stat.countMAE_CloseATR;

if (avgMFE != EMPTY_VALUE &&
    avgMAE != EMPTY_VALUE &&
    avgMAE > 0.0)
{
    mfeMaeRatio = avgMFE / avgMAE;
}

string line = "";

AppendField(line, patternType);
AppendField(line, AgeBasisName(ageBasis));
AppendField(line, scopeSymbol);
AppendField(line, scopeTimeframe);
AppendField(line, ZoneGroupName(zoneGroup));
AppendField(line, AgeBucketName(ageBucket));
AppendField(line, touchBucket);
AppendField(line, depthBucket);
AppendField(line, approachBucket);
AppendField(line, heightBucket);
AppendField(line, sessionBucket);

AppendField(line, IntegerToString(stat.total));
AppendField(
    line,
    stat.total >= MinReliableSamples ? "YES" : "NO"
);
AppendField(line, IntegerToString(MinReliableSamples));

AppendField(line, IntegerToString(stat.localReversalFirst));
AppendField(line, IntegerToString(stat.localBreakoutFirst));
AppendField(line, IntegerToString(stat.localTimeout));
AppendField(line, IntegerToString(stat.localOther));
AppendField(line, IntegerToString(localDecisive));
AppendField(line, PercentOrBlank(stat.localReversalFirst,
stat.total));
AppendField(line, PercentOrBlank(stat.localBreakoutFirst,
stat.total));
AppendField(line, PercentOrBlank(stat.localTimeout,
stat.total));
AppendField(line, PercentOrBlank(stat.localReversalFirst,
localDecisive));
AppendField(line, WilsonLower95OrBlank(
stat.localReversalFirst, localDecisive));

AppendField(line, IntegerToString(stat.primaryReversal));
AppendField(line, PercentOrBlank(stat.primaryReversal,
stat.total));
AppendField(line, IntegerToString(stat.localBreakout));

```

```

AppendField(line, PercentOrBlank(stat.localBreakout,
stat.total));

AppendField(line, IntegerToString(stat.sequenceReversalOnly
));
AppendField(line, IntegerToString(stat.sequenceBreakoutOnly
));
AppendField(line, IntegerToString(
stat.sequenceReversalThenBreakout));
AppendField(line, IntegerToString(
stat.sequenceBreakoutThenReversal));
AppendField(line, IntegerToString(stat.sequenceNone));
AppendField(line, IntegerToString(stat.sequenceOther));

AppendField(line, IntegerToString(stat.reached05ATR));
AppendField(line, PercentOrBlank(stat.reached05ATR,
stat.total));
AppendField(line, IntegerToString(stat.reached10ATR));
AppendField(line, PercentOrBlank(stat.reached10ATR,
stat.total));
AppendField(line, IntegerToString(stat.reached15ATR));
AppendField(line, PercentOrBlank(stat.reached15ATR,
stat.total));
AppendField(line, IntegerToString(stat.reached20ATR));
AppendField(line, PercentOrBlank(stat.reached20ATR,
stat.total));
AppendField(line, IntegerToString(stat.reached1Height));
AppendField(line, PercentOrBlank(stat.reached1Height,
stat.total));
AppendField(line, IntegerToString(stat.reached2Height));
AppendField(line, PercentOrBlank(stat.reached2Height,
stat.total));
AppendField(line, IntegerToString(stat.reached3Height));
AppendField(line, PercentOrBlank(stat.reached3Height,
stat.total));

AppendField(line, IntegerToString(
stat.structuralOppositeFirst));
AppendField(line, IntegerToString(
stat.structuralSameTypeFirst));
AppendField(line, IntegerToString(stat.structuralTimeout));
AppendField(line, IntegerToString(
stat.structuralTargetsBroken));
AppendField(line, IntegerToString(
stat.structuralEndOfHistory));
AppendField(line, IntegerToString(stat.structuralOther));
AppendField(line, IntegerToString(structuralDecisive));
AppendField(line, PercentOrBlank(
stat.structuralOppositeFirst, stat.total));
AppendField(line, PercentOrBlank(
stat.structuralSameTypeFirst, stat.total));

```

```

AppendField(line, PercentOrBlank(
stat.structuralOppositeFirst, structuralDecisive));
AppendField(line, WilsonLower95OrBlank(
stat.structuralOppositeFirst, structuralDecisive));

AppendField(line, DoubleOrBlank(avgMFE, 6));
AppendField(line, DoubleOrBlank(avgMAE, 6));
AppendField(line, DoubleOrBlank(mfeMaeRatio, 6));
AppendField(
    line,
    AverageOrBlank(
        stat.sumMFE_CloseHeights,
        stat.countMFE_CloseHeights,
        6
    )
);
AppendField(
    line,
    AverageOrBlank(
        stat.sumMAE_CloseHeights,
        stat.countMAE_CloseHeights,
        6
    )
);

AppendField(
    line,
    AverageOrBlank(
        stat.sumBarsPrimaryReversal,
        stat.countBarsPrimaryReversal,
        4
    )
);
AppendField(
    line,
    AverageOrBlank(
        stat.sumBarsLocalBreakout,
        stat.countBarsLocalBreakout,
        4
    )
);
AppendField(
    line,
    AverageOrBlank(
        stat.sumBarsStructuralReversal,
        stat.countBarsStructuralReversal,
        4
    )
);
AppendField(
    line,
    AverageOrBlank(

```

```

        stat.sumBarsStructuralBreakout,
        stat.countBarsStructuralBreakout,
        4
    )
);

AppendField(
    line,
    AverageOrBlank(
        stat.sumAgeMinutes,
        stat.countAgeMinutes,
        4
    )
);

AppendField(
    line,
    AverageOrBlank(
        stat.sumAgeBars,
        stat.countAgeBars,
        4
    )
);

AppendField(
    line,
    AverageOrBlank(
        stat.sumTouchNumber,
        stat.countTouchNumber,
        4
    )
);

AppendField(
    line,
    AverageOrBlank(
        stat.sumDepthPercent,
        stat.countDepthPercent,
        4
    )
);

AppendField(
    line,
    AverageOrBlank(
        stat.sumApproachNetMoveATR5,
        stat.countApproachNetMoveATR5,
        6
    )
);

AppendField(
    line,
    AverageOrBlank(
        stat.sumApproachRangeATR5,
        stat.countApproachRangeATR5,
        6
    )
);

```

```

    )
);
AppendField(
    line,
    AverageOrBlank(
        stat.sumZoneHeightATR,
        stat.countZoneHeightATR,
        6
    )
);
AppendField(
    line,
    AverageOrBlank(
        stat.sumTouchBodyATR,
        stat.countTouchBodyATR,
        6
    )
);
AppendField(
    line,
    AverageOrBlank(
        stat.sumTouchRelativeVolume,
        stat.countTouchRelativeVolume,
        6
    )
);

FileWriteString(handle, line + "\r\n");
}

```

```
//+-----
```

```
///| Сохранение age_statistics.csv
```

```
//+-----
bool SaveAgeStatistics()
{
    ResetLastError();

    int handle = FileOpen(
        OutputAgeStatisticsFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка создания ",
            OutputAgeStatisticsFileName,
            ". Код ошибки: ",
            GetLastError()
        );
    }
}

```

```

    );

    ResetLastError();
    return(false);
}

WriteStatisticsHeader(handle);

for(int basis = 0; basis < AGE_BASE_COUNT; basis++)
{
    for(int zone = 0; zone < ZONE_GROUP_COUNT; zone++)
    {
        for(int age = 0; age < AGE_BUCKET_COUNT; age++)
        {
            int index = BaseAgeIndex(basis, zone, age);

            WriteStatisticsRow(
                handle,
                "AGE_ONLY",
                basis,
                zone,
                age,
                "ALL",
                "ALL",
                "ALL",
                "ALL",
                "ALL",
                g_ageStats[index]
            );
        }
    }
}

FileFlush(handle);
FileClose(handle);

string fullPath =
    TerminalInfoString(TERMINAL_DATA_PATH) +
    "\\MQL4\\Files\\" +
    OutputAgeStatisticsFileName;

Print("Файл создан: ", fullPath);
return(true);
}

```

```
//+-----
```

```
//| Сохранение pattern_statistics.csv
```

```
//+-----
```

```
bool SavePatternStatistics()
```

```

{
    ResetLastError();

    int handle = FileOpen(
        OutputPatternStatisticsFileName,
        FILE_WRITE | FILE_TXT | FILE_ANSI
    );

    if(handle == INVALID_HANDLE)
    {
        Print(
            "Ошибка создания ",
            OutputPatternStatisticsFileName,
            ". Код ошибки: ",
            GetLastError()
        );

        ResetLastError();
        return(false);
    }

    WriteStatisticsHeader(handle);

    for(int basis = 0; basis < AGE_BASE_COUNT; basis++)
    {
        for(int zone = 0; zone < ZONE_GROUP_COUNT; zone++)
        {
            for(int age = 0; age < AGE_BUCKET_COUNT; age++)
            {
                for(int touch = 0; touch < TOUCH_BUCKET_COUNT;
touch++)
                {
                    WriteStatisticsRow(
                        handle,
                        "AGE_TOUCH",
                        basis,
                        zone,
                        age,
                        TouchBucketName(touch),
                        "ALL",
                        "ALL",
                        "ALL",
                        "ALL",
                        g_ageTouchStats[
                            AgeTouchIndex(
                                basis,
                                zone,
                                age,
                                touch
                            )
                        ]
                    );
                }
            }
        }
    }
}

```

```

    }

    for(int depth = 0; depth < DEPTH_BUCKET_COUNT;
depth++)
    {
        WriteStatisticsRow(
            handle,
            "AGE_DEPTH",
            basis,
            zone,
            age,
            "ALL",
            DepthBucketName(depth),
            "ALL",
            "ALL",
            "ALL",
            g_ageDepthStats[
                AgeDepthIndex(
                    basis,
                    zone,
                    age,
                    depth
                )
            ]
        );
    }

    for(int touch2 = 0; touch2 < TOUCH_BUCKET_COUNT;
touch2++)
    {
        for(int depth2 = 0; depth2 < DEPTH_BUCKET_COUNT;
depth2++)
        {
            WriteStatisticsRow(
                handle,
                "AGE_TOUCH_DEPTH",
                basis,
                zone,
                age,
                TouchBucketName(touch2),
                DepthBucketName(depth2),
                "ALL",
                "ALL",
                "ALL",
                g_ageTouchDepthStats[
                    AgeTouchDepthIndex(
                        basis,
                        zone,
                        age,
                        touch2,
                        depth2
                    )
                ]
            );
        }
    }
}

```

```

        ]
    );
}
}

for(int approach = 0;
    approach < APPROACH_BUCKET_COUNT;
    approach++)
{
    WriteStatisticsRow(
        handle,
        "AGE_APPROACH_5_BARS",
        basis,
        zone,
        age,
        "ALL",
        "ALL",
        ApproachBucketName (approach) ,
        "ALL",
        "ALL",
        g_ageApproachStats[
            AgeApproachIndex(
                basis,
                zone,
                age,
                approach
            )
        ]
    );
}

for(int height = 0;
    height < HEIGHT_BUCKET_COUNT;
    height++)
{
    WriteStatisticsRow(
        handle,
        "AGE_ZONE_HEIGHT",
        basis,
        zone,
        age,
        "ALL",
        "ALL",
        "ALL",
        HeightBucketName (height) ,
        "ALL",
        g_ageHeightStats[
            AgeHeightIndex(
                basis,
                zone,
                age,
                height
            )
        ]
    );
}

```

```

        )
    ]
    );
}

for(int session = 0;
    session < SESSION_BUCKET_COUNT;
    session++)
{
    WriteStatisticsRow(
        handle,
        "AGE_SESSION",
        basis,
        zone,
        age,
        "ALL",
        "ALL",
        "ALL",
        "ALL",
        SessionBucketName(session),
        g_ageSessionStats[
            AgeSessionIndex(
                basis,
                zone,
                age,
                session
            )
        ]
    );
}
}
}
}

FileFlush(handle);
FileClose(handle);

string fullPath =
    TerminalInfoString(TERMINAL_DATA_PATH) +
    "\\MQL4\\Files\\" +
    OutputPatternStatisticsFileName;

Print("Файл создан: ", fullPath);
return(true);
}

```

//+-----

//| Проверка входных параметров

//+-----

```

bool ValidateInputs()
{
    if(StringLen(InputTouchesFileName) == 0 ||
        StringLen(OutputAgeStatisticsFileName) == 0 ||
        StringLen(OutputPatternStatisticsFileName) == 0)
    {
        Print("Ошибка: имена файлов не могут быть пустыми.");
        return(false);
    }

    if(MinReliableSamples < 1)
    {
        Print("Ошибка: MinReliableSamples должен быть больше 0."
);
        return(false);
    }

    if(AsiaEndHour < 1 ||
        AsiaEndHour > 23 ||
        LondonEndHour <= AsiaEndHour ||
        LondonEndHour > 23 ||
        NewYorkEndHour <= LondonEndHour ||
        NewYorkEndHour > 23)
    {
        Print("Ошибка: неверно заданы границы торговых сессий.");
        return(false);
    }

    if(ProgressEveryRows < 0)
    {
        Print(
"Ошибка: ProgressEveryRows не может быть отрицательным.");
        return(false);
    }

    return(true);
}

```

//+-----

//| Итоговый журнал

//+-----

```

void PrintSummary()
{
    int localDecisive =
        g_globalStats.localReversalFirst +
        g_globalStats.localBreakoutFirst;

    int structuralDecisive =
        g_globalStats.structuralOppositeFirst +

```

```

        g_globalStats.structuralSameTypeFirst;

        Print(
"=====
");
        Print("БЛОК 03. ИТОГ СТАТИСТИЧЕСКОЙ ГРУППИРОВКИ");
        Print(
"=====
");
        Print("Прочитано строк          : ", g_rowsRead);
        Print("Включено правильных касаний      : ", g_rowsIncluded);
        Print("Пропущено WRONG_SIDE/REENTRY      : ", g_skippedWrongSide);
        Print("Пропущено другого графика         : ", g_skippedOtherChart);
        Print("Пропущено некорректных строк      : ", g_skippedInvalid);
        Print(
"-----
");
        Print("REVERSAL_FIRST          : ",
g_globalStats.localReversalFirst);
        Print("BREAKOUT_FIRST         : ",
g_globalStats.localBreakoutFirst);
        Print("LOCAL TIMEOUT          : ",
g_globalStats.localTimeout);

        if(localDecisive > 0)
        {
            Print(
                "Разворот среди решённых, %      : ",
                DoubleToString(
                    100.0 * g_globalStats.localReversalFirst /
                    localDecisive,
                    2
                )
            );
        }

        Print(
"-----
");
        Print("OPPOSITE_ZONE_FIRST      : ",
g_globalStats.structuralOppositeFirst);
        Print("SAME_TYPE_ZONE_FIRST     : ",
g_globalStats.structuralSameTypeFirst);
        Print("STRUCTURAL TIMEOUT      : ",
g_globalStats.structuralTimeout);

        if(structuralDecisive > 0)
        {
            Print(
                "Противоположная зона, %      : ",
                DoubleToString(
                    100.0 * g_globalStats.structuralOppositeFirst /
                    structuralDecisive,
                    2
                )
            );
        }
    }
}

```

```

        )
    );
}

Print(
"=====
");
}

//+-----

//| Точка входа скрипта

//+-----
void OnStart()
{
    if(!ValidateInputs())
        return;

    if(!InitializeStatistics())
    {
        Print(
"Ошибка: не удалось выделить память для статистики.");
        return;
    }

    Print("Блок 03: чтение ", InputTouchesFileName, "...");

    if(!LoadAndAggregateTouches())
        return;

    if(g_rowsIncluded <= 0)
    {
        Print(
"Ошибка: в анализ не вошло ни одного правильного касания.");
        return;
    }

    if(!SaveAgeStatistics())
        return;

    if(!SavePatternStatistics())
        return;

    PrintSummary();
}

//+-----

```